PRIVATE

Code Assessment

of the Intmax 2 Smart Contracts

October 30, 2024

Produced for

INTMAX

by

CHAINSECURITY

Contents

1 Executive Summary 3
2 Assessment Overview 5
3 Limitations and use of report 12
4 Terminology 13
5 Findings 14
6 Resolved Findings 15
7 Informational 26
8 Notes 28

1 Executive Summary

Dear Intmax team,

Thank you for trusting us to help Intmax with this security audit. Our executive summary provides an overview of subjects covered in our audit of the latest reviewed contracts of Intmax 2 according to Scope to support you in forming an opinion on their security risks.

Intmax implements a zk-rollup for private transfers. Liquidity is on/offboarded on Ethereum while the state of the rollup is managed on Scroll.

The most critical subjects covered in our audit are general liveness of the rollup, denial of services, and user assets safety. Security regarding all subjects is good.

The other general subjects covered are correct usage of Scroll Messenger, access control, and global interaction between the contracts. Security regarding all the aforementioned subjects is high.

In summary, we find that the codebase provides a high level of security.

Given the small amount of tests in the codebase, we strongly recommend enhancing the testing suite as several issues found in the audit could have been caught by more extensive testing.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,

ChainSecurity

1.1 Overview of the Findings

Below we provide a brief numerical overview of the findings and how they have been addressed.

Critical-Severity Findings	5
• Code Corrected	5
High-Severity Findings	0
Medium-Severity Findings	2
• Code Corrected	1
• Risk Accepted	1
Low-Severity Findings	6
• Code Corrected	5
• Specification Changed	1

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

The assessment was performed on the source code files inside the Intmax 2 repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received.

V	Date	Commit Hash	Note
1	16 September 2024	b809c890a74f762f72b5fbbe41d768fe1e77711e	Initial Version
2	13 October 2024	3c983747e823df38e60630334023edb3f3b60ffe	Version 2
3	22 October 2024	2e77bbbe77a5b86449b588691183b05d9887603b	Version 3

For the solidity smart contracts, the compiler version `0.8.27` was chosen.

The following directories were included in the scope of the assessment:

- `contracts/block-builder-registry/`: Only in Version 1.
- `contracts/common/`
- `contracts/contribution/`
- `contracts/liquidity/`
- `contracts/rollup/`
- `contracts/withdrawal/`

2.1.1 Excluded from scope

Anything not listed in the scope is considered out of scope for this assessment. This includes but is not limited to:

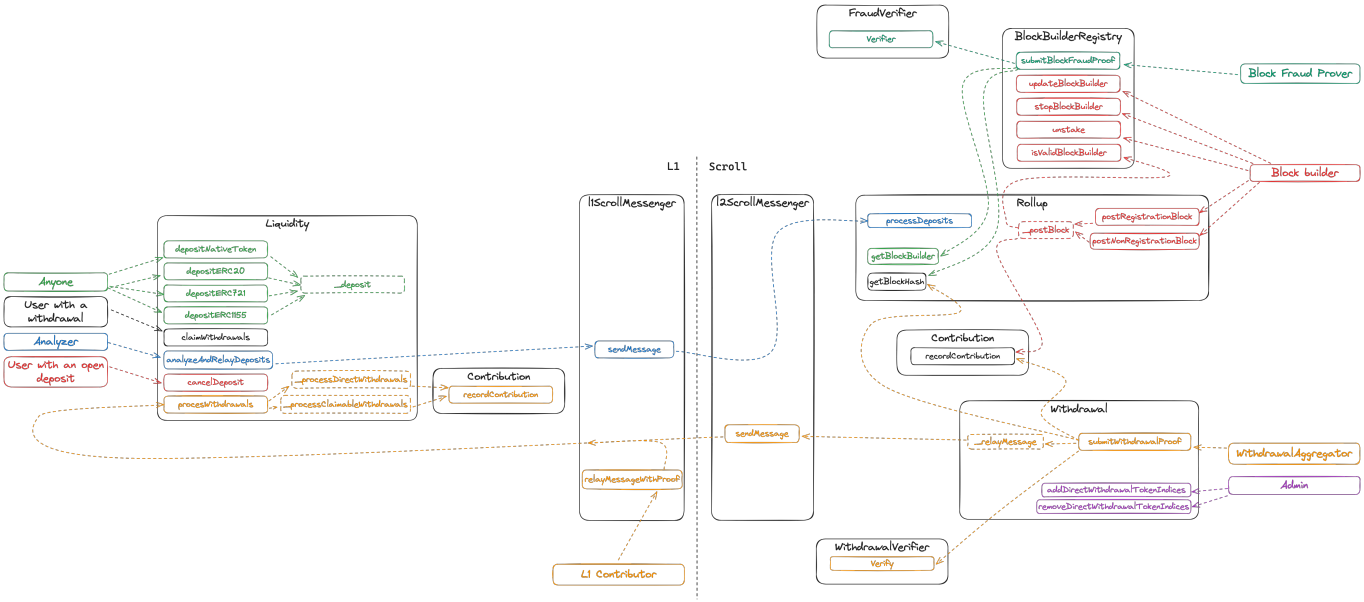
- `contracts/test/`
- The plonk verifier contracts the system interact with.
- The Scroll Messenger contracts.
- Dependencies used in the project such as `openzeppelin-contracts` or `prb-math`.
- As of Version 2, contracts in `contracts/block-builder-registry/` are no longer used by the system and are considered out of scope.

2.2 System Overview

This system overview describes the initially received version (Version 1) of the contracts as defined in the Assessment Overview.

Furthermore, in the findings section, we have added a version icon to each of the findings to increase the readability of the report.

Intmax 2 is a zk-rollup for private transfers. Liquidity is on/offboarded on Ethereum while the state of the rollup is managed on Scroll.



2.2.1 Common Components

2.2.1.1 Contribution

The `Contribution` contract is a simple contract used to record user contributions. It does not distribute rewards but only accounts for contribution and is supposed to be used by a reward system to incentivize users to perform various action in the system. The contract works per period, for each period, a set of tags are mapped to weights. Tags represent the different actions that can be performed by users, and weights represent the amount of contribution that each action represents. The contract is controlled by a trusted entity, the weight registrar, holding `WEIGHT_REGISTRAR` role, that can begin a new period using `incrementPeriod()` and add tags and their respective weight for some given period by calling `registerWeights()`.

The `CONTRIBUTOR` role should be assigned to the Intmax's contracts that should notify the `Contribution` contract of the actions performed by users. Using the function `recordContribution()`, the contract can record the contribution of a user for a given tag and amount in the current period.

Rewarding mechanisms that are out of scope of this audit should be implemented in a separate contract that would interact with the `Contribution` contract to retrieve the contribution data using `getContributionRate()` which gives the ratio of the contribution of a user in a given period compared to the total contribution of the period.

2.2.2 Ethereum Smart Contracts

2.2.2.1 Liquidity

The Liquidity contract is the main entry point for the users to interact with the system. It allows users to deposit and withdraw funds.

Deposits

The deposit process is divided in a request/accept pattern. The user first requests a deposit and later the `ANALYZER` controlled by Intmax, either accepts or rejects the deposits in batches.

The contract accepts native ETH as well as any `ERC20`, `ERC721` and/or `ERC1155` tokens. Users can call the respective deposit function to open a deposit, they should specify `recipientSaltHash` which is a salted hash of the recipient address:

- `depositNativeToken`: For ETH deposits.
- `depositERC20`: For ERC20 deposits.
- `depositERC721`: For ERC721 deposits.
- `depositERC1155`: For ERC1155 deposits.

Upon depositing, if the deposited token was not already registered, the system creates a token Index for the token. In case of Native and ERC20 tokens, there will only be one token index for each token. In case of ERC721 and ERC1155 tokens, a fresh token index is created for each token ID.

Deposits are placed internally in a queue that the `ANALYZER` can iterate over in batches using `analyzeAndRelayDeposits()`. The function iterates over the amount of specified deposits and either rejects them, or accepts them depending on the specific parameters. Rejected deposits can be withdrawn (canceled) by the user, while for accepted deposits, a message is relayed to the `Rollup` contract on Scroll through the `ScrollMessenger` (Note here that the funds are not relayed and stay in the Liquidity contract).

Canceling a deposit is always possible as long as the deposit has not been both analyzed and accepted by the `ANALYZER` (in this case the deposit is accounted for in the `Rollup`).

Withdrawals

Withdrawals are relayed from the `Withdrawal` contract on Scroll when a valid withdrawal proof is submitted, to the `Liquidity` contract by calling the function `withdrawal.processWithdrawals()`. The call is bridged from Scroll to Ethereum via the `ScrollMessenger`. `processWithdrawals` receives both:

- A list of direct withdrawals that are to be processed, where funds are sent directly to the users.
- A list of hashes of claimable withdrawals that can later be claimed by the users by calling the function `claimWithdrawals` and providing the pre-image of concerned hashed withdrawals.

2.2.2.2 Contribution

Because the way the `ScrollMessenger` is implemented, any upcoming transactions from `L2` need to be finalized using the `relayMessageWithProof()` function on the Scroll Messenger contract on `L1`. This means that when the `Withdrawal` contracts on Scroll calls the `Liquidity` contract on Ethereum, someone has to finalize the transaction on Ethereum, and pay for the gas. To incentivize this, the `Liquidity` contract records contributions using the `Contribution` contract.

On Ethereum, only the `Liquidity` contract is given the `Contribution`'s `contributor` role, and can record contributions.

It is assumed that for each period, the tags available are:

- `PROCESS_DIRECT_WITHDRAWALS`
- `PROCESS_CLAIMABLE_WITHDRAWALS`

For each new withdrawal batch being processed using `processWithdrawals()`, the `Liquidity` contract records the following in favor of the `tx.origin`:

- `N` contributions for the tag `PROCESS_DIRECT_WITHDRAWALS` where `N` is the number of direct withdrawals being processed.

- `M` contributions for the tag `PROCESS_CLAIMABLE_WITHDRAWALS` where `M` is the number of claimable withdrawals being processed.

2.2.3 Scroll Smart Contracts

2.2.3.1 BlockBuilderRegistry

The `BlockBuilderRegistry` is intended to be interacted with two type of entities:

1. The block builders, that must stake 0.1 ETH to be able to post blocks.
2. The block fraud provers, that can submit fraud proofs against invalid blocks, to slash the corresponding block builder.

This system is needed as the `Rollup` contract does not check the validity of the blocks, and optimistically assumes that the blocks are valid.

The block builders can register or increase their stake using the function `updateBlockBuilder()`. There is no limit on how much they can stake, but at least 0.1 ETH is required to be able to post blocks. The function `stopBlockBuilder()` can be called to start a Challenge duration. During that period, the block builder cannot post new blocks, and the block fraud provers can submit fraud proofs against the previous blocks, this is needed to avoid block builder posting invalid blocks and immediately withdrawing their stake to avoid slashing. Finally, after the challenge period, the block builder can withdraw their stake using the function `unstake`.

The block fraud provers can submit fraud proofs using the function `submitBlockFraudProof()`. They must provide the public inputs consisting of the block number, the block hash and the challenger together with a proof. The contract will check:

1. The provided `blockHash` and `blockNumber` are valid regarding the chain stored in `Rollup`.
2. The message sender matches the provided `challenger` to avoid frontrunning.
3. The proof is valid using some onchain Plonk proof verifier that is out of scope of this audit.
4. The block was not already slashed.

In case all the checks pass, the block builder is slashed for that block:

- If the block builder had a stake greater than or equal to the minimum stake, half of the `MIN_STAKE_AMOUNT` goes to the fraud prover, and the other half is burned.
- If the block builder had a stake smaller than `MIN_STAKE_AMOUNT / 2`, all stake goes to the fraud prover.
- Otherwise, `MIN_STAKE_AMOUNT / 2` goes to the fraud prover and the rest is burned.

In the last two cases, the builder stake could be below the minimum required if for example the builder was slashed multiple times. In the first and second case, part of the stake is burned to avoid the builder to be able to fraud prove their own blocks.

The `Rollup` contract interacts with the `BlockBuilderRegistry` to check that block builders trying to post new blocks are registered and have enough stake to post a block by calling `isValidBlockBuilder()`.

2.2.3.2 Rollup

The `Rollup` contract has two main functions:

1. Processing deposits from the `Liquidity` contract on Ethereum. 1. Handling and including new blocks from block builders.

The function `processDeposits()` can only be called by the `ScrollMessenger` when a message is relayed from the `Liquidity` contract on Ethereum. The function is given a list of deposit hashes and simply includes them in the `depositTree`, a sparse Merkle tree containing all deposits. The function also updates the `depositTreeRoot` given the updated tree.

Block builders that are valid according to the `BlockBuilderRegistry` can submit new blocks using the functions `postRegistrationBlock()` and `postNonRegistrationBlock()`. The contract checks that the aggregated public key matches the aggregated signature provided, however, the contract does not check that the block is valid, invalid blocks will be ignored by the Zero Knowledge part of the system.

Finally, the contract stores the hash of the new block which consists of:

- The previous block hash.
- The current `depositTreeRoot`.
- The signature hash.
- The block number.

Where the signature hash is itself the hash of the following fields:

- `isRegistrationBlock`: Whether the block is for account updates (transfers sent by accounts already existing in the account tree) or account registrations (transfers sent by accounts not yet existing in the account tree).
- `txTreeRoot`: The root of the transaction tree.
- `senderFlags`: A bit-field indicating which senders have signed the transaction tree.
- `publicKeysHash`: The hash of the aggregated public keys of the senders.
- `accountIdHash`: The hash of the aggregated account IDs of the senders.
- `aggregatedPublicKey`: The aggregated public key of the senders.
- `aggregatedSignature`: The aggregated signature of the senders.
- `messagePoint`: The point corresponding to the `txTreeRoot` mapped to a point in the `G2` group of the `BN254` curve.

2.2.3.3 Withdrawal

The `Withdrawal` contract is responsible for processing withdrawals. It can receive public inputs, a withdrawal list together with a proof from some unpermissioned "withdrawal aggregators". If the proof is valid, the function relays the set of withdrawals to the `Liquidity` contract on Ethereum using the `ScrollMessenger`. For a call to `submitWithdrawalProof()`, to be successful, the contract checks whether given a list of withdrawals, the public inputs `lastWithdrawalHash` and `withdrawalAggregator` together with a proof:

1. The set of provided withdrawals, when hashed in chain, matches the provided public input `lastWithdrawalHash`.
2. The public input `withdrawalAggregator` matches the message sender to avoid frontrunning.
3. The provided proof is valid using some onchain Plonk proof verifier that is out of scope of this audit.
4. For each withdrawal:
    1. The withdrawal's `nullifier` has not been used already to avoid double spending.
    2. The withdrawal's block number and hash are valid regarding the chain stored in `Rollup`.

Withdrawals can be either direct or claimable depending on the token being withdrawn. Direct withdrawals will be pushed to receivers by the `Liquidity` contract, while claimable withdrawals will be

available for the receivers to claim by calling the `Liquidity` contract. The `owner` of the `Withdrawal` contract can update which tokens are directly withdrawable or claimable using the functions `addDirectWithdrawalTokenIndices()` and `removeDirectWithdrawalTokenIndices()`.

2.2.3.4 Contribution

On Scroll, both the `Rollup` and `Withdrawal` contracts are assumed to be given the `Contribution`'s `contributor` role, and can record contributions.

It is assumed that for each period, the tags available are:

- `WITHDRAWAL`
- `POST_BLOCK`

For each new withdrawal proof being submitted using `submitWithdrawalProof()`, the `Withdrawal` contract records `N` contribution with the tag `WITHDRAWAL` in favor of the `msg.sender`, where `N` is the number of withdrawals being processed. Similarly, for each new block posted (either registration or non-registration block) the `Rollup` contract records `1` contribution with the tag `POST_BLOCK` in favor of the `msg.sender`.

2.2.4 Trust Model and Roles

Roles

- The `Liquidity`'s `ANALYZER` is a trusted entity that is controlled by Intmax.
- The `Contributor`'s `WEIGHT_REGISTRAR` is a trusted entity that is controlled by Intmax.
- Every contract is upgradable by some owner or privileged admin role, we assume that this party is a trusted entity that will not perform any malicious upgrade.

Trust Assumptions

- It is assumed that the deployment of the system is done correctly and that all contract correctly reference each other.
- The system can handle at most `2**32-1` token index and should not be used for more than that.
- On Ethereum, the `Liquidity` contract is given the `CONTRIBUTOR` role on the `Contribution` contract.
- On Scroll, both the `Rollup` and `Withdrawal` contracts are given the `CONTRIBUTOR` role on the `Contribution` contract.
- For all period, each of the `Contributor` contracts has the tags required by the `Liquidity` contract on Ethereum, or the `Rollup` and `Withdrawal` contracts on Scroll.
- Both `BlockBuilderRegistry.submitBlockFraudProof()` and `Withdrawal.submitWithdrawalProof()` are marked as `onlyOwner`, for this audit. We, however, assume that they can be called by anyone as Intmax indicated that the checks will be removed for the production version. This is reflected in the system overview above and in the findings section.
- The Plonk verifier contracts are assumed to work as expected and are treated as a black box for this review. It is assumed that the provided proofs are always correctly verified if correct, or rejected if incorrect.
- The `ScrollMessenger` is assumed to work as expected and is treated as a black box for this review. It is assumed that the messages are always correctly relayed between the two chains.
- The `ANALYZER` is assumed to carefully provide enough `msg.value` and `gasLimit` when calling `analyzeAndRelayDeposits()`. To cover for the Scroll transaction. He should also make sure that the transaction on Scroll cannot result in a Proof overflow.

- We assume that the `ANALYZER` will carefully only accept deposits with ERC-compliant tokens that are not malicious and present no specific risks. Moreover, it is assumed that the system does not support rebasing, fee-on-transfer tokens as well as more generally token that transfer different balances that the amount given to `transfer()` or `transferFrom()`.
- The `owner` of the `Withdrawal` contract is assumed to carefully set the direct withdrawal tokens to avoid any potential issues with the direct withdrawal mechanism.
- The `burn` address of the `BlockBuilderRegistry` is assumed to be chosen such that it never reverts upon receiving funds.

2.2.5 Changes in Version 2

All changes in the Version 2 are fixes for the issues found during the audit. The only significant change is the removal of the `BlockBuilderRegistry` in favor of a `RateLimiterLib`.

As raised in Slashing mechanism is not effective to prevent spamming blocks, the staking/slashing mechanism implemented by the `BlockBuilderRegistry` was not effective to prevent spamming blocks that could result in halting the chain. This mechanism was removed in favor of a `RateLimiterLib` that enforces a penalty for block builders that post blocks too fast.

Given:

- $\alpha = \frac{1}{3}$
- `k = 0.001 eth`
- `targetInterval = 15s`

The contract keeps an exponential moving average of the interval between block produced which is computed as $emaInterval = \alpha * interval + (1 - \alpha) * emaInterval$. And the penalty paid to the system for posting a block too fast (if $targetInterval > emaInterval$) is computed as follows: $penalty = k * (targetInterval - emaInterval)^2$.

Note that this implies that:

1. Block producers no longer have to stake.
2. Producing an invalid blocks is no longer punished by the system.
3. It is assumed that transactions fees will adapt to the congestion of the chain to adapt for penalty to be paid by the block builders.
4. It is assumed that block validity proving will be able to keep up with the block producing rate.

A function `withdrawPenaltyFee()` was additionally added to the `Rollup` contract to allow the owner to withdraw the penalty fees paid by the block builders.

2.2.6 Changes in Version 3

All changes in the Version 2 are fixes for the issues found during the audit. The only significant change is the is that `ERC721` and `ERC1155` tokens are no longer supported as direct withdrawal tokens and can only be claimed in a two step process.

3 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.

4 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- *Likelihood* represents the likelihood of a finding to be triggered or exploited in practice
- *Impact* specifies the technical and business-related consequences of a finding
- *Severity* is derived based on the likelihood and the impact

We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	Impact		
	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

5 Findings

In this section, we describe any open findings. Findings that have been resolved have been moved to the Resolved Findings section. The findings are split into these different categories:

- Design: Architectural shortcomings and design inefficiencies
- Correctness: Mismatches between specification and implementation

Below we provide a numerical overview of the identified findings, split up by their severity.

Critical-Severity Findings	0
High-Severity Findings	0
Medium-Severity Findings	1

- Contribution Mechanism Can Be Abused Risk Accepted

Low-Severity Findings	0

5.1 Contribution Mechanism Can Be Abused

Design Medium Version 2 Risk Accepted

CS-INTMAX_SC-023

The `Contribution` contracts are supposed to account for action made by users that are helping the system. It is expected to be used by some reward mechanism that will incentivize user to perform such actions.

- Contribution recorded when submitting a withdrawal proof can be abused given that there is no limit on how many proofs can be created for dummy withdrawals. Anyone could spam the system with dummy (valid) proofs to get large amount of contribution accounted for.
- Block posting is not limited, and the contribution does not depend on the content of the block, in case posting a block at some time `t` would not result in a penalty, anyone could post an empty block to get a contribution accounted for.

Risk accepted:

Intmax has accepted the risks of this issue.

6 Resolved Findings

Here, we list findings that have been resolved during the course of the engagement. Their categories are explained in the Findings section.

Below we provide a numerical overview of the identified findings, split up by their severity.

Critical-Severity Findings	5

- Block Builder Can Be Unslashable Code Corrected
- Block Builder Can Dodge Slashing Code Corrected
- Chain Will Halt Code Corrected
- Recipient Can DOS Withdrawals Batch Code Corrected
- Slashing Mechanism Is Not Effective to Prevent Spamming Blocks Code Corrected

High-Severity Findings	0

Medium-Severity Findings	1

- Incorrect Variable Used Code Corrected

Low-Severity Findings	6

- Bad Naming of Function isChallengeDuration Code Corrected
- Deposit Queue's Front Can Decrease Code Corrected
- Ignored Return Value Code Corrected
- Misleading Comment Specification Changed
- Missing Sanity Checks Code Corrected
- Shadowed Variable Code Corrected

Informational Findings	8

- Boolean Equality Checks Code Corrected
- Broad Function Visibility Code Corrected
- Frontrunning Deposits Code Corrected
- Gas Optimizations Code Corrected
- Implementations Can Be Initialized Code Corrected
- Missing Events Code Corrected
- AccessControlUpgreadable Initializer Code Corrected
- isDepositOngoing Specifications Code Corrected

6.1 Block Builder Can Be Unslashable

Design Critical Version 1 Code Corrected

CS-INTMAX_SC-001

When submitting a valid fraud proof, the corresponding block builder gets slashed. The function `_slashBlockBuilder()` performs the following:

1. If the block builder has no stake, the call reverts.
2. If the block builder has less than 0.1 ETH stake, and is marked as valid, the block builder's status is set to invalid.
3. The block builder's stake is decreased with `min(builder's stake, 0.1 ETH)`.

In parallel, block builders can post new blocks as long as they are marked as valid, independently of their stake.

This pattern presents two issues:

- Because step 2 happens before step 3, a block builder, who has been slashed and has now less than 0.1 ETH stake, can still post new blocks, and will only be marked as invalid after their next slashing provided they don't add more stake.
- In the case the block builders get slashed, and their whole stake is removed, they can still post new blocks, as they are still marked as valid, but it is impossible to slash them again given that the step 1 above will revert upon any call trying to slash them. This means that the block builder can post invalid blocks forever without any risk of being slashed again.

Code corrected

The `BlockBuilderRegistry` is no longer used by the system and is considered out of scope as of Version 2. This issue is no longer relevant although still present in the contract.

6.2 Block Builder Can Dodge Slashing

Design | Critical | Version 1 | Code Corrected

CS-INTMAX_SC-002

When a block builder wants to unstake, they should follow these steps:

1. Call `stopBlockBuilder()` which marks the block builder as invalid to prevent them from posting new blocks, and start a challenge duration by setting the block builder's `stopTime` to the current time.
2. Wait for the challenge duration to end (1 day).
3. Call `unstake()` which checks that `block.timestamp - info.stopTime >= CHALLENGE_DURATION` to get their stake back.

Because `info.stopTime` is by default `0`, a block builder can dodge the challenge duration by simply calling `unstake()`. This would allow him to unstake right after posting an invalid block and dodge any potential slashing.

Similarly, in case a block builder already called `stopBlockBuilder()`, he can call `updateBlockBuilder()` with `callvalue=0` to reset the `stopTime` to `0` and then call `unstake()` to dodge the challenge period.

Code corrected

The `BlockBuilderRegistry` is no longer used by the system and is considered out of scope as of Version 2. This issue is no longer relevant although still present in the contract.

6.3 Chain Will Halt

Correctness Critical Version 1 Code Corrected

CS-INTMAX_SC-003

The functions `BlockHashLib.getBlockNumber` and `BlockHashLib.getPrevHash` take as input `blockHashes` in memory. However, when being called by `Rollup._postBlock()`, for example, the `blockHashes` is provided as a storage variable. This means that during execution, the entire array is loaded from storage to memory. As the Intmax chain will grow, the array will grow as well, and it finally reaches a point where the array will be too large to be loaded without reaching Scroll gas limit (10 million gas). According to estimations, at most `5000` blocks are needed to reach the gas limit. It should be noted that the limit can be reached faster if the chain is spammed with new blocks.

All call location of the functions `getBlockNumber` and `getPrevHash` are subject to this issue.

Code corrected

Both `getBlockNumber()` and `getPrevHash()` functions are updated to take `blockHashes` as a storage variable.

6.4 Recipient Can DOS Withdrawals Batch

Design Critical Version 1 Code Corrected

CS-INTMAX_SC-004

In `Withdrawal`, the `owner` can set token indices as `direct withdrawal`, meaning that upon receiving a message from the `Withdrawal` contract through the `ScrollMessenger`, the `Liquidity` contract, will push the tokens to the recipients. When the function `processWithdrawals()` is called by `L1ScrollMessenger.relayMessageWithProof()`, if pushing such a direct withdrawal token fails, the whole batch will be reverted. This behavior can be used by malicious actors to perform a denial-of-service attack.

Because the withdrawals of the batch are already registered in the `Withdrawal` contract and only the `L1` transaction is reverting, the only way to recover from this attack is would be to upgrade the `Liquidity` contract to a new version that include a fix.

Below are examples of why pushing a direct withdrawal token can fail:

- `ETH`: The recipient is a contract that reverts on receiving the call.
- `ERC20` token: The recipient is some address that makes the transfer fail (e.g some ERC20 tokens revert when transferring to `address(0)`)
- `ERC20` token: For some tokens like USDC, it could be that the recipient is blacklisted, and the transfer fails.
- `ERC1155` or `ERC721` token: The recipient is `address(0)`.
- `ERC1155` token: The recipient is a contract that does not implement the `onERC1155Received` function.

Code corrected

In Version 3, direct withdrawals are still pushed to the recipient, but in case a transfer fails, the revert is not propagated to the whole batch. Instead, the withdrawal that caused the failed transfer is marked as failed and made claimable so that the recipient can later withdraw it, and the batch continues to be processed. Additionally, support for direct withdrawals of ERC721 and ERC1155 tokens was removed and according to Intmax:

```
If ERC721 or ERC1155 tokens are input into _processDirectWithdrawal, they will be changed to claimable withdrawals.
For ERC20 tokens, we have modified the implementation based on OpenZeppelin's safeTransfer.sol,
using low-level contract calls to prevent reverts caused by returndatasize checks.
```

6.5 Slashing Mechanism Is Not Effective to Prevent Spamming Blocks

Design | Critical | Version 1 | Code Corrected

CS-INTMAX_SC-005

The slashing mechanism is designed to prevent block builders from spamming the chain and putting pressure on the block validity prover that might not be able to handle the load. However, the current implementation of slashing is not effective to prevent this behavior:

1. There is no rate limit on the block producing, a malicious block producer can produce a large number of blocks in a short period of time and get slashed only once for those.
2. It is possible to produce valid, but useless blocks, and not get slashed. Spamming such blocks would still put pressure on the system and the block producer would not be punished for it.

In the case block validity provers are not able to handle the load and cannot keep up with the block producing rate, the system would become unusable for honest users.

Code corrected

The slashing mechanism was removed from the system and replaced with a rate limiter on block producing.

6.6 Incorrect Variable Used

Correctness | Medium | Version 1 | Code Corrected

CS-INTMAX_SC-006

In `Contribution.getContributionRate()`, the following is done:

```
for (uint256 i = 0; i < allTags[periodNumber].length; i++) {
    bytes32 tag = allTags[currentPeriod][i];
    ...
```

In the case the current period has a different amount of tags or different tags than the requested period, calling the function leads to unexpected behaviors.

- If `allTags[periodNumber].length` is greater than `allTags[currentPeriod].length`, the function will revert.
- If both periods have different tags, the function will not revert but might return an incorrect value.

Code corrected

The function now indexes `allTags` with `periodNumber` instead of `currentPeriod`.

6.7 Bad Naming of Function `isChallengeDuration`

Correctness | Low | Version 1 | Code Corrected

CS-INTMAX_SC-007

The function `isChallengeDuration()`'s name is misleading as it checks that since the `stopTime`, at least `CHALLENGE_DURATION` seconds has passed. Hence, it actually returns true if the challenge duration is over and returns false if still in the challenge duration.

Code corrected

The function `isChallengeDuration()` has been renamed to `hasChallengeDurationPassed()` to better reflect its purpose.

6.8 Deposit Queue's Front Can Decrease

Design | Low | Version 1 | Code Corrected

CS-INTMAX_SC-008

In `Liquidity`, the `ANALYZER` can call `analyzeAndRelayDeposits` and specify the `upToDepositId` parameter to limit the number of deposits to analyze the deposit queue. This limit is checked in `DepositQueue.analyze` to be strictly smaller than the queue's `rear`. However, it is not enforced to be greater than or equal to the queue's `front`. If the `analyzer` specifies `upToDepositId` smaller than the current queue's `front`, and an empty list of `rejectDepositIds`, `front` will be updated to `upToDepositId`, effectively breaking the assumption that the queue's `front` can never decrease. This could lead to unexpected behavior in the system as deposits could then be analyzed and accepted multiple times.

Code corrected

In `DepositQueue.analyze()`, it is now ensured that `upToDepositId` is greater than or equal to the queue's `front`.

6.9 Ignored Return Value

Design | Low | Version 1 | Code Corrected

CS-INTMAX_SC-009

In `Withdrawal.initialize()` the return value of `directWithdrawalTokenIndices.add()` is ignored. In case `_directWithdrawalTokenIndices` contains twice the same value, the execution will not revert as it would for `addDirectWithdrawalTokenIndices()`.

Code corrected

The return value of `directWithdrawalTokenIndices.add()` is now checked, and the function will revert if the value is already present.

6.10 Misleading Comment

Correctness | Low | Version 1 | Specification Changed

CS-INTMAX_SC-010

In `BlockBuilderRegistry`, `_slashBlockBuilder` contains the following misleading comment:

```
if (info.stakeAmount < MIN_STAKE_AMOUNT) {
    // The Block Builder cannot post a block unless it has a minimum amount of stakes,
    // so it does not normally enter into this process.
    ...
```

A block builder with an initial stake of `0.15` ETH could end up in this situation for example if they posted two invalid blocks before being slashed for both. The first slashing will reduce their stake to `0.05` ETH, and the second slashing will trigger the condition.

Specification changed

The comment was updated to reflect the possibility of entering the branch as described above.

6.11 Missing Sanity Checks

Design | Low | Version 1 | Code Corrected

CS-INTMAX_SC-011

Several sanity checks are missing in the code. The following are some examples:

- `_scrollMessenger`, `_contribution`, `_rollup`, `_liquidity` and `_withdrawalVerifier` are not checked for zero address in `Withdrawal.initialize()`.
- `_scrollMessenger`, `_contribution`, `_blockBuilderRegistry` and `_liquidity` are not checked for zero address in `Rollup.initialize()`.
- `_rollup` and `_fraudVerifier` are not checked for zero address in `BlockBuilderRegistry.initialize()`.
- `_rollup`, `_l1ScrollMessenger`, `_withdrawal`, `_analyzer` and `_contribution` are not checked for zero address in `Liquidity.initialize()`.

Code corrected

All missing sanity checks mentioned above have been added to the codebase.

6.12 Shadowed Variable

Design | Low | Version 1 | Code Corrected

CS-INTMAX_SC-012

In `BlockBuilderRegistry`, the local variable `owner` of the function `_slashBlockBuilder` shadows the storage variable `owner`. This greatly reduce the readability of the code and can lead to introducing issues.

Code corrected

The local variable `owner` is renamed to `_owner`.

6.13 Boolean Equality Checks

Informational | Version 1 | Code Corrected

CS-INTMAX_SC-013

In several occurrences, boolean variables are compared with `true` or `false`. This is not necessary as the variable itself is already a boolean value. The following list shows the functions containing occurrences of this issue:

- `Liquidity.isDepositOngoing()`.
- `Liquidity.canCancelDeposit()`.
- `Withdrawal.submitWithdrawalProof()`.
- `Withdrawal.addDirectWithdrawalTokenIndices()`.
- `Withdrawal.removeDirectWithdrawalTokenIndices()`.
- `BlockBuilderRegistry.isStaking()`.
- `BlockBuilderRegistry.unstake()`.
- `BlockBuilderRegistry._slashBlockBuilder()`.

Code corrected

The boolean variables are now directly used in the comparison.

6.14 Broad Function Visibility

Informational | Version 1 | Code Corrected

CS-INTMAX_SC-014

For functions that are expected to be called from other contracts only, the function visibility can be restricted to `external` instead of `public`. This applies to:

- `Rollup.initialize()`
- `Liquidity.initialize()`
- `Withdrawal.initialize()`
- `BlockBuilderRegistry.initialize()`

- `Contribution.initialize()`

For functions that are not expected to be called from other contracts, the visibility can be restricted from `public` to `internal`. This applies to:

- `TokenData.__TokenData_init()`

Code corrected

The visibility of the functions mentioned above has been updated to restrict access to the appropriate level.

6.15 Frontrunning Deposits

Informational | Version 1 | Code Corrected

CS-INTMAX_SC-015

In `Liquidity`, the `recipientSaltHash` passed to `depositERC20`, `depositERC721`, and `depositERC1155` are always checked to be unique. A malicious user could consistently frontrun deposit transactions with the same `recipientSaltHash` and a dummy token to temporarily prevent other users from depositing tokens. As this assumes that the transactions are visible to the attacker in advanced, it could be easily mitigated by the use of private mempools. Such griefing attack is costly and not profitable for the attacker and only serves to disrupt the system.

Code corrected

The system was changed to check for duplicate `depositHash` instead of `recipientSaltHash`. As a result, frontrunning a deposit is now equivalent to making the actual deposit on behalf of the user. It is expected that in such cases, the frontend will make it clear to the user that the deposit is then already ongoing.

6.16 Gas Optimizations

Informational | Version 1 | Code Corrected

CS-INTMAX_SC-016

The following gas savings were identified in the Version 1 of the codebase:

Liquidity

1. In `onlyWithdrawal()`, `withdrawal` is loaded from storage twice, it could be cached.
2. In `onlyWithdrawal()`, `l1ScrollMessenger` is loaded from storage twice, it could be cached.

DepositQueueLib

1. `depositQueue.rear` is always equivalent to the length of the `depositData` and could be removed.
2. In `enqueue()`, `depositQueue.rear` is loaded from storage twice, it could be cached.
3. In `analyze()`, `depositQueue.front` is loaded at each iteration of the first loop, it could be cached.

TokenData

1. In `_createTokenIndex()`, `nextTokenIndex` is loaded from storage twice, it could be cached.
2. `nextTokenIndex` is always equivalent to the length of the `tokenInfoList` and could be removed.
3. `getNativeTokenIndex()` can simply return 0, as native token is always initialized with this index. Same argument holds for `getTokenIndex` when `tokenType` is `TokenType.NATIVE`.

Contribution

1. In `getWeights()`, `allTags[periodNumber].length` is loaded from storage at each iteration of the loop.
2. In `recordContribution`, `currentPeriod` is loaded from storage twice, it could be cached.
3. `getContributionRate()` can convert to `UD60x18` at the end just before the division.

DepositTreeLib

1. The function `initialize` initializes several storage variables to zero, which is unnecessary.
2. In `deposit()`, `depositTree.depositCount` is loaded from storage twice, it could be cached.

Rollup

1. In `onlyLiquidityContract()`, `l2ScrollMessenger` is loaded from storage twice, it could be cached.
2. In `processDeposits()`, `lastProcessedDepositId` is loaded from storage twice, it could be cached.
3. In `processDeposits()`, The result of `depositTree.getRoot()` could be cached to avoid loading `depositTreeRoot` from storage after updating it.
4. In `_postBlock()`, `depositTreeRoot` is loaded from storage twice, it could be cached.

BlockBuilderRegistry

1. In `stopBlockBuilder()`, the entire `BlockBuilderInfo` is loaded to memory to only update two fields, which could be optimized.
2. In `getValidBlockBuilders()`, `blockBuilderAddresses.length` is loaded from storage at each iteration of each of the loops.
3. In `getValidBlockBuilders()`, `blockBuilderAddresses[i]` is loaded from storage twice at each iteration of the second loop it could be cached.

4. `updateBlockBuilder()` gets `url` as a string in memory. As the function is defined external, its location can be defined in calldata. 5.

Withdrawal

In `submitWithdrawalProof()`, `lastDirectWithdrawalId` (resp. `lastClaimableWithdrawalId`) is updated and read for each withdrawal, this could be avoided using the local variables `directWithdrawalIndex` and `claimableWithdrawalIndex`. Additionally, both storage variables are read twice after the for-loop iteration while cached variables could be used instead.

As of Version 2, the following gas savings were identified in the codebase:

Contribution

1. In `incrementPeriod()`, `currentPeriod` is loaded from storage twice, it could be cached.

Code corrected

All gas optimizations raised in the Version 1 & Version 2 of the codebase have been addressed.

6.17 Implementations Can Be Initialized

Informational | Version 1 | Code Corrected

CS-INTMAX_SC-017

Every contract in the codebase are `UUPSUpgradeable`. However, `_disableInitializers()` is never called in their constructors, this means that the implementations can be initialized by untrusted parties.

In the current state of the code, it does not impose a security issue as no contract contains the `SELFDESTRUCT` or `DELEGATECALL` opcodes (except for the `UUPSUpgradable` contract itself that however protect `upgradeToAndCall()` behind an `onlyProxy` modifier to mitigate a *vulnerability* found in older versions of openzeppelin contracts).

However, it is recommended to call `_disableInitializers()` in the constructor of the contracts as a best practice and to prevent future issues in case the codebase is updated in the future with the aforementioned opcodes.

Code corrected

The function `_disableInitializers()` is now called in the constructor of all contracts.

6.18 Missing Events

Informational | Version 1 | Code Corrected

CS-INTMAX_SC-019

As external observers rely on events to track the state of the contract, it is recommended to emit events when important state changes occur.

1. The contract `Contribution` does not emit any events.
2. In `Withdrawal`, `addDirectWithdrawalTokenIndices()` and `removeDirectWithdrawalTokenIndices()` do not emit events.

Code corrected

Both `Contribution` and `Withdrawal` now emit events when important state changes occur.

6.19 `AccessControlUpgreadable` Initializer

Informational | Version 1 | Code Corrected

CS-INTMAX_SC-021

In `Liquidity`, `initialize()` does not call `AccessControlUpgradeable.__AccessControl_init()` as opposed to

`Contribution.initialize()` for example. Although the access control initializer is empty, it is a good practice to initialize all the parent contracts.

Code corrected

`Liquidity` now calls `AccessControlUpgradeable.__AccessControl_init()` in its `initialize()` function.

6.20 `isDepositOngoing` Specifications

Informational **Version 1** **Code Corrected**

CS-INTMAX_SC-025

`isDepositOngoing` will always return true after the analyzer accepted it, although the name of the function suggest that it should return false after the deposit is completed.

Code corrected

The function `isDepositOngoing` has been renamed to `isDepositValid` to better reflect its purpose.

7 Informational

We utilize this section to point out informational findings that are less severe than issues. These informational issues allow us to point out more theoretical findings. Their explanation hopefully improves the overall understanding of the project's security. Furthermore, we point out findings which are unrelated to security.

7.1 Rate Limiter Design Limitations

Informational | Version 2 | Acknowledged

CS-INTMAX_SC-024

The Rate limiter is set up with a target interval of `15` seconds which correspond to the expected time to create a validity proof for a block.

- It should be noted that in case of high demand for block space, it could be that by paying penalties, blocks are created more frequently than the target interval, in which case, the validity proof producer would not be able to keep up with the block producing rate.
- This could be even more problematic if a malicious party is willing to pay some penalty to make this behavior happen. For example, given the current implementation, it will cost at most `0.001` ETH per block to consistently post a block every `14` seconds instead of `15` seconds. Meaning that over one day, the validity proof producer would receive `6171` blocks where it has the throughput to only produce `5760`, which would lead the last block posted during that day to be handled by the validity proof producer `102` minutes after it was posted. This would cost 6.171 eth per day to the malicious party, which could be partially covered by contributions rewards or transactions fees from the posted blocks.

Acknowledged:

Intmax has acknowledged this issue.

7.2 Locked Funds

Informational | Version 1 | Acknowledged

CS-INTMAX_SC-018

In case tokens are directly sent to any of the system's contract by mistake, there is no way to recover the funds. This is especially the case for the `Liquidity` contract, which is the main entry point for users.

Acknowledged

Intmax acknowledged the risk of locked funds.

7.3 Withdrawal Order

Informational | Version 1 | Acknowledged

CS-INTMAX_SC-020

In `Liquidity`, `_processDirectWithdrawals()` and `_processClaimableWithdrawals()` emit events containing respectively the last processed direct or claimable withdrawal ID. However, nothing enforces that messages are relayed in order on Ethereum, and it could be that an event for a higher ID is processed before an event for a lower ID.

Acknowledged

This behavior has been acknowledged by Intmax.

7.4 `transfer()` Usage

Informational **Version 1** **Acknowledged**

CS-INTMAX_SC-022

In multiple instances, the system utilizes the Solidity `.transfer()` built-in function to transfer ETH.

Using `transfer()` should be avoided because gas costs are variable and can change over time. Since gas costs are not fixed, contracts should not rely on specific and hardcoded gas amounts which is the case as `transfer()` is dependent on a fixed gas amount of 2300, which can lead to issues if gas costs change in the future.

Acknowledged

The issue was acknowledged by Intmax.

8 Notes

We leverage this section to highlight further findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require an immediate modification inside the project. Instead, they should raise awareness in order to improve the overall understanding.

8.1 Direct Withdrawals Can Be Forced to Be Claimable Instead

Note **Version 2**

By providing some very specific amount of gas when calling `ScrollMessenger.relayMessageWithProof()`, the transaction sender can force some withdrawal to be claimable instead of being directly withdrawn. This can be done by setting a specific amount of gas that would make the transfer for the direct withdrawal fail, but still allows the rest of the transaction to be executed without running out of gas.

8.2 Gas Expensive Functions

Note **Version 1**

Several functions in the code base will eventually become extremely gas expensive and could require more than the chain's block gas limit to execute. It is assumed that the following functions are to be called only by off-chain services:

- `BlockBuilderRegistry.getValidBlockBuilders()`.
- `Withdrawal.getDirectWithdrawalTokenIndices()`.

8.3 Non-fungible Tokens Transfer Privacy

Note **Version 1**

`ERC721` (and `ERC1155` tokens when some of its token IDs are non-fungible tokens) are supported by the system, it should be noted that as soon as the token is withdrawn from the Intmax system, external party can easily link the address that deposited the token into the system with the address that withdrew it as they are unique.

8.4 Non-standard Tokens

Note **Version 1**

In `Liquidity`, the functions `depositERC20`, `depositERC721`, and `depositERC1155` can accept any contracts as tokens even non-compliant ones. The `analyzer` should take special care to only accept deposits whose tokens are standard and compliant with the corresponding `ERC` to avoid unexpected behavior in the system. The following are example of tokens that could lead to unexpected behavior:

- Rebasing tokens.
- Fee on transfer tokens.

- Tokens such as `cUSDCv3` that can transfer different amounts of token that the one specified (In this case transfers the whole balance of the sender if `type(uint256).max` is passed as the amount).

Moreover, the owner of `Withdrawal`, should take care to only set direct withdrawals for tokens that:

- Are either the native token or standard and compliant with `ERC20`.
- Do not fit in one of the categories above.
- Do not have callbacks, or any way to give the execution flow to a malicious actor when being called by `Liquidity._processDirectWithdrawal()`.
- Do not contain any way of executing an `invalid` opcode, or, more generally, consuming all the gas forwarded to them.
- Have been accepted by the analyzer.

8.5 Replaying Deposits

Note **Version 1**

As mentioned in the System Overview, we assumed that the user holding the `ANALYZER` role, must correctly set `gasLimit` when calling `analyzeAndRelayDeposits()`. However, if for any reason, the `ANALYZER` does not specify a sufficient `gasLimit` to cover for the Scroll execution gas costs, sending the deposit message from Ethereum to Scroll would not be completed. Although Scroll messenger on Ethereum devises the capability of replaying a message by the function `replayMessage()`, users are not incentivized to call this as no contribution belongs to retrying deposits.

Furthermore, if the messenger fails to forward the message to Scroll, anyone can call `dropMessage()`. In the current state of code, calling `dropMessage()` reverts as it requires the message sender (in our case, the `Liquidity` contract) to implement the `onDropMessage()` function. However, enough care should be taken if Intmax decides to implement this functionality.

8.6 `blockBuilderAddresses` Can Have Duplicates

Note **Version 1**

In `BlockBuilderRegistry`, the `blockBuilderAddresses` can contain duplicate addresses.